



1. Problem Title & Link

Title: LeetCode 547 – Number of Provinces

Link: <https://leetcode.com/problems/number-of-provinces/>

2. Problem Statement (Short Summary)

There are n cities.

You are given an $n \times n$ adjacency matrix `isConnected`, where:

- `isConnected[i][j] = 1` $\rightarrow$ City i and City j are directly connected.
- `isConnected[i][j] = 0` $\rightarrow$ No direct connection.

A **province** is a group of directly or indirectly connected cities.

Return the **number of provinces**.

3. Examples (Input $\rightarrow$ Output)

Example 1

Input

`isConnected =`

```
[
  [1,1,0],
  [1,1,0],
  [0,0,1]
]
```

Output

2

Explanation

Province 1

0 ----- 1

Province 2

2

Example 2

Input

`isConnected =`



```
[  
  [1,0,0],  
  [0,1,0],  
  [0,0,1]  
]
```

Output

3

4. Constraints

- $1 \leq n \leq 200$
- $n == \text{isConnected.length}$
- $\text{isConnected}[i][i] == 1$
- Matrix is symmetric.

5. Core Concept (Pattern / Topic)

- Graph Traversal
- DFS
- BFS
- Connected Components
- Union Find

6. Thought Process (Step-by-Step Explanation)**Observation**

Every city is a graph node.

If

$\text{isConnected}[i][j] == 1$

there is an edge.

Example

0 ---- 1

2

There are two connected components.

What are we counting?

We are **NOT** counting cities.

We are counting

Connected Components



Every connected component equals one province.

Algorithm

Loop through every city.

If the city has not been visited,

New Province Found

Increase answer.

Perform DFS/BFS.

Visit every city in this province.

Continue scanning.

Why DFS?

Starting from one city,

DFS reaches every city that belongs to the same province.

Therefore,

that province is counted exactly once.

Why This Works

Every unvisited city starts a new connected component.

DFS marks the entire component.

No city is counted twice.

7. Visual / Intuition Diagram (ASCII)

```
Example
[
  [1,1,0],
  [1,1,0],
  [0,0,1]
]
Graph
0 ----- 1

2
Traversal
Start 0

↓

Visit 1

Province = 1
```



↓

City 2 unvisited

Province = 2

Answer

2

8. Pseudocode (Language Independent)

```
visited = {}  
  
province = 0  
  
for every city  
    if not visited  
        province++  
        DFS(city)  
  
return province
```

9. Code Implementation

Python (DFS)

```
class Solution:  
  
    def findCircleNum(self, isConnected):  
  
        n = len(isConnected)  
        visited = set()  
  
        def dfs(city):  
  
            visited.add(city)  
  
            for neighbor in range(n):  
  
                if (  
                    isConnected[city][neighbor] == 1  
                    and neighbor not in visited  
                ):  
                    dfs(neighbor)  
  
        provinces = 0
```



```
for city in range(n):  
  
    if city not in visited:  
  
        provinces += 1  
        dfs(city)  
  
return provinces
```

Python (BFS)

```
from collections import deque  
  
class Solution:  
  
    def findCircleNum(self, isConnected):  
  
        n = len(isConnected)  
  
        visited = set()  
  
        provinces = 0  
  
        for city in range(n):  
  
            if city in visited:  
                continue  
  
            provinces += 1  
  
            queue = deque([city])  
  
            visited.add(city)  
  
            while queue:  
  
                node = queue.popleft()  
  
                for neighbor in range(n):  
  
                    if (  
                        isConnected[node][neighbor] == 1  
                        and neighbor not in visited  
                    ):  
  
                        visited.add(neighbor)  
                        queue.append(neighbor)  
  
        return provinces
```



Java (DFS)

```
class Solution {  
  
    public int findCircleNum(int[][] isConnected) {  
  
        int n = isConnected.length;  
  
        boolean[] visited = new boolean[n];  
  
        int provinces = 0;  
  
        for(int city = 0; city < n; city++){  
  
            if(!visited[city]){  
  
                provinces++;  
  
                dfs(isConnected, visited, city);  
            }  
        }  
  
        return provinces;  
    }  
  
    private void dfs(int[][] graph,  
                    boolean[] visited,  
                    int city){  
  
        visited[city] = true;  
  
        for(int neighbor = 0;  
            neighbor < graph.length;  
            neighbor++){  
  
            if(  
                graph[city][neighbor] == 1 &&  
                !visited[neighbor]  
            ){  
  
                dfs(graph, visited, neighbor);  
            }  
        }  
    }  
}
```

10. Time & Space Complexity

Time Complexity

 $O(n^2)$

For every city,

we scan the entire row of the adjacency matrix.

Space Complexity

DFS

 $O(n)$

Visited array + recursion stack.

11. Common Mistakes / Edge Cases

- Treating the matrix like an adjacency list.
- Forgetting to mark visited.
- Counting every city instead of every connected component.
- Revisiting already explored cities.

12. Detailed Dry Run

Example

```
[  
  [1,1,0],  
  [1,1,0],  
  [0,0,1]  
]
```

Visited

{}

City 0

Not visited.

Province = 1

DFS

0

↓

1

Visited

{0,1}

Continue

City 1



Already visited.

Skip.

City 2

Not visited.

Province = 2

DFS

2

Visited

{0,1,2}

Answer

2

13. Common Use Cases (Real-Life / Interview)

- Social network communities.
- Friend groups.
- Computer network clusters.
- Airline connectivity.
- Connected component detection.

14. Common Traps (Important!)

- Forgetting that the graph is represented as an **adjacency matrix**, not an adjacency list.
- Using only direct neighbors without exploring indirect connections.
- Not marking nodes as visited before recursive calls.
- Counting connected cities instead of connected components.

15. Builds To (Related LeetCode Problems)

- LC 1971 – Find if Path Exists in Graph
- LC 841 – Keys and Rooms
- LC 200 – Number of Islands
- LC 695 – Max Area of Island
- LC 684 – Redundant Connection
- LC 207 – Course Schedule

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
DFS	$O(n^2)$	$O(n)$	Most common interview solution



BFS	$O(n^2)$	$O(n)$	Same complexity
Union-Find (DSU)	$O(n^2 \cdot \alpha(n))$	$O(n)$	Excellent for dynamic connectivity problems

17. Why This Solution Works (Short Intuition)

Each province is a connected component in the graph. When we encounter an unvisited city, we have found a new province. DFS/BFS visits every city connected to it (directly or indirectly), marking them as visited. Since each connected component is explored exactly once, the number of DFS/BFS starts equals the number of provinces.

18. Variations / Follow-Up Questions

1. Return the cities belonging to each province instead of just the count.
2. Find the size of the largest province.
3. Solve the problem using Union-Find (Disjoint Set Union).
4. Given roads added dynamically, update the number of provinces after each addition.
5. Convert an adjacency list representation into a province count.
- 6.